

SZEGEDI TUDOMÁNYEGYETEM

Természettudományi és Informatikai Kar

Informatikai Intézet

Szakdolgozat

ProdCty - audiofókuszú közösségi platform producereknek és zenészeknek

Tompa Márton

Gazdaságinformatikus BSc

Témavezető: Dr. Bilicki Vilmos

egyetemi docens

Szeged

2026

Tartalmi összefoglaló

A szakdolgozat célja egy audiofókuszú közösségi webalkalmazás, a ProdCty megtervezése és megvalósítása. A rendszer producereknek, zenészeknek és zenei alkotóknak biztosít olyan online felületet, ahol hanganyagokat tölthetnek fel, böngészhetnek, meghallgathatnak, értékelhetnek, valamint visszajelzést adhatnak egymás munkáira. A projekt kiindulópontja az volt, hogy a zenei alkotók számára hasznos lehet egy olyan platform, amely nem videós vagy vizuális tartalmakra épít, hanem kifejezetten az audio tartalmat helyezi a felhasználói élmény középpontjába.

A fejlesztés során egy kliens-szerver architektúrájú webalkalmazás készült. A frontend React és TypeScript alapokon valósult meg, míg a backend Node.js és Express keretrendszerre épül. Az adatok tárolásához MongoDB adatbázis és Mongoose objektummodellező könyvtár került felhasználásra, a felhasználói azonosítás pedig JWT alapú hitelesítéssel történik. A rendszer fő funkciói közé tartozik a regisztráció és bejelentkezés, a hanganyagok feltöltése és listázása, a sample és demo tartalmak elkülönített kezelése, az audiolejátszás, a letöltés, a kommentelés, az értékelés, a jelentés, az együttműködési kérés, a profilkezelés, valamint az adminisztrátori moderáció.

A dolgozat bemutatja a projekt célkitűzéseit, a funkcionális és nem funkcionális követelményeket, az alkalmazott technológiákat, a rendszer architektúráját, az adatmodellt, a főbb backend- és frontend-megoldásokat, valamint a tesztelés és validáció folyamatát. Külön fejezet foglalkozik a mesterséges intelligencia fejlesztés során betöltött szerepével, annak előnyeivel, korlátaival és a validálás módjával. A projekt eredménye egy működő MVP, amely további fejlesztések, például fejlettebb ajánlórendszer, objektumtár-alapú médiatárolás vagy részletesebb moderációs folyamatok alapjául szolgálhat.

Kulcsszavak: webalkalmazás, audio platform, React, Node.js, MongoDB, közösségi alkalmazás

Tartalom

Tartalmi összefoglaló	2
1. Bevezetés	4
2. Irodalmi és technológiai áttekintés	5
2.1 Hasonló rendszerek és különbségek	6
3. Célkitűzés.....	7
4. Felhasznált eszközök és technológiák.....	8
4.1 Frontend technológiák.....	8
4.2 Backend technológiák	8
4.3 Adattárolás és hitelesítés	8
4.4 Fejlesztési és ellenőrzési eszközök	9
5. Alkalmazott módszerek és rendszerterv	10
5.1 Architektúra	10
5.2 Követelmények	10
5.3 Adatmodell.....	11
5.4 API-felépítés	11
5.5 Biztonsági és adatvédelmi szempontok	12
6. Eredmények és megvalósítás	13
6.1 Backend megvalósítás.....	13
6.2 Frontend megvalósítás	13
6.3 Sample és demo tartalmak elkülönítése	14
6.4 Közösségi funkciók.....	14
6.5 Adminisztrátori funkciók	14
7. Tesztelés és értékelés	15
7.1 Manuális tesztelési folyamat.....	15
7.2 Ismert korlátok	15
8. Mesterséges intelligencia használata a fejlesztés során	17

9. Összefoglalás	20
Irodalomjegyzék.....	21
Nyilatkozat	22

1. Bevezetés

A digitális zenei alkotás az elmúlt években egyre szélesebb körben elérhetővé vált. A producerek, zenészek és hangmérnökök ma már otthoni környezetben is képesek professzionális vagy félprofesszionális hanganyagok létrehozására. Ezzel párhuzamosan megnőtt az igény olyan online platformokra, ahol az alkotók megoszthatják munkáikat, visszajelzést kaphatnak, inspirációt gyűjthetnek, illetve hangmintákat találhatnak saját projektjeikhez.

A meglévő közösségi platformok jelentős része elsősorban vizuális vagy videós tartalmakra épül. Bár ezek alkalmasak zenei tartalmak megosztására is, a felhasználói élmény gyakran nem az audio tartalom köré szerveződik. Egy zenei producer számára például nem feltétlenül a vizuális megjelenés, hanem a hanganyag gyors meghallgatása, értékelése, letöltése vagy véleményezése a legfontosabb. Ebből az igényből indult ki a ProdCty projekt.

A ProdCty egy audiofókuszú közösségi platform producereknek és zenészeknek. A rendszer célja, hogy elkülönített felületet biztosítson a különböző típusú hanganyagok számára. A projekt koncepciójában külön szerepet kapnak a felhasználók által feltöltött demók, amelyek elsősorban visszajelzés céljából jelennek meg, valamint a sample jellegű hanganyagok, amelyek újrafelhasználásra és letöltésre is alkalmasak lehetnek. A két tartalomtípus elkülönítése nemcsak felhasználói felületi kérdés, hanem jogosultsági, adatmodellbeli és üzleti logikai szempontból is lényeges tervezési feladat.

A szakdolgozat célja a ProdCty webalkalmazás megtervezésének és megvalósításának bemutatása. A dolgozat nem egy teljes értékű, piaci használatra kész zenei szolgáltatás létrehozását tűzi ki célul, hanem egy működő MVP, vagyis minimálisan életképes termék elkészítését. Az MVP célja, hogy demonstrálja az alkalmazás fő működési logikáját, a felhasználói folyamatokat, az audio tartalmak kezelését és a közösségi interakciók alapjait.

2. Irodalmi és technológiai áttekintés

A dolgozat témája elsősorban alkalmazásfejlesztési jellegű, ezért a szakirodalmi áttekintés a projektben használt technológiák, architekturális minták és webes fejlesztési megoldások rövid bemutatására koncentrál. A ProdCty megvalósítása modern webalkalmazási modellre épül, ahol a felhasználói felület, a szerveroldali üzleti logika és az adattárolás elkülönített rétegekben jelenik meg.

A frontend oldalon a React könyvtár komponensalapú fejlesztési megközelítést biztosít. A komponensek újrafelhasználható felületi egységek, amelyek saját felelősségi körrel rendelkeznek. Ez jól illeszkedik a ProdCty jellegéhez, mivel az alkalmazás több ismétlődő felületi elemet használ, például track-kártyákat, audiolejátszókat, űrlapokat, profilblokkokat és adminisztrációs nézeteket. A TypeScript használata a JavaScripthez képest statikus típusellenőrzést ad, amely segíti a hibák korai felismerését és a nagyobb kódrendszer fenntarthatóságát [1], [2].

A backend oldalon a Node.js és az Express keretrendszer biztosítja az API-k működését. A Node.js eseményvezérelt futtatási környezet, amely gyakran használt választás webes API-k, valós idejű vagy médiaorientált alkalmazások fejlesztésénél. Az Express minimalista webes keretrendszer, amely route-ok, middleware-ek és HTTP-válaszok kezelésével támogatja az alkalmazás szerveroldali szervezését [3], [4].

Az adattárolás MongoDB és Mongoose segítségével történik. A MongoDB dokumentumalapú adatbázis, amely rugalmasan kezeli a változó szerkezetű adatokat. Egy közösségi audio platform esetén ez előnyös lehet, mivel a felhasználók, feltöltések, kommentek, értékelések, jelentések és együttműködési kérések eltérő struktúrájú, de egymással kapcsolatban álló adatok. A Mongoose sémákat, validációt és modelleket biztosít a Node.js alkalmazás számára [5], [6].

A felhasználói azonosítás JWT alapú tokenes hitelesítéssel történik. A JSON Web Token olyan kompakt formátum, amely alkalmas arra, hogy a backend a bejelentkezés után aláírt tokenet adjon a felhasználónak, a frontend pedig ezt a későbbi kérésekben azonosításra használja [7]. A biztonság szempontjából fontos, hogy a tokenhez kapcsolódó titkos kulcs ne kerüljön publikus repóba, és hogy az érzékeny konfigurációk külön környezeti változóknak szerepeljenek.

2.1 Hasonló rendszerek és különbségek

A ProdCty koncepciója több létező digitális zenei platform problémateréhez kapcsolódik. A sample könyvtárak jellemzően hangminták keresésére, meghallgatására és letöltésére koncentrálnak, míg a közösségi média típusú rendszerek elsősorban a tartalmegosztást és reakciókat helyezik előtérbe. A ProdCty ezekből a mintákból indul ki, de az MVP-ben nem egy teljes értékű üzleti szolgáltatás másolása volt a cél, hanem az audiofókuszú interakciók és a felhasználók által generált hanganyagok kezelésének bemutatása.

A rendszer egyik sajátossága, hogy különbséget tesz sample és demo tartalmak között. A sample jellegű hanganyagok inkább letölthető, újrafelhasználható elemekként jelennek meg, míg a demo tartalmak fő célja a visszajelzés, értékelés és együttműködési lehetőség. Ez a különbségtétel a projekt egyik központi szakmai döntése, mert nemcsak a felületen, hanem az adatmodellben, az API-végpontokban és a jogosultsági szabályokban is megjelenik.

3. Célkitűzés

A szakdolgozat célja egy olyan működő webalkalmazás elkészítése, amely bemutatja egy audiofókuszú közösségi platform alapvető működését. A cél nem egy teljes értékű, üzletileg skálázott termék létrehozása volt, hanem egy fejleszthető MVP elkészítése, amely a legfontosabb felhasználói folyamatokat és rendszertervezési döntéseket demonstrálja.

A fejlesztés során a következő célok kerültek meghatározásra:

- regisztráció és bejelentkezés megvalósítása JWT alapú hitelesítéssel;
- sample és demo típusú hanganyagok feltöltésének, listázásának és lejátszásának támogatása;
- a sample és demo tartalmak eltérő szabályainak megjelenítése a felületen és a backend logikában;
- kommentelési, értékelési, szavazási és jelentési funkciók biztosítása;
- felhasználói profil és publikus profilnézet kialakítása;
- együttműködési kérés küldésének és kezelésének támogatása;
- adminisztrátori moderációs felület létrehozása problémás tartalmak és felhasználók kezelésére;
- működő README, futtatási útmutató, dokumentáció és tesztelési alap létrehozása.

A projekt során kiemelt cél volt, hogy a dokumentáció és a megvalósítás összhangban legyen egymással. Ez különösen azért fontos, mert a kezdeti tervekben több olyan fejlettebb funkció is szerepelt, amely az MVP jelenlegi állapotában még nem célja a rendszernek. Ilyen például a teljes értékű ajánlórendszer, a professzionális objektumtár-alapú médiaarchitektúra, a valós idejű kollaboráció vagy mélyebb audioanalitikai modulok. Ezek a későbbi fejlesztések között szerepelhetnek, de a jelen dolgozatban a ténylegesen elkészült funkciók bemutatása a lényeg.

4. Felhasznált eszközök és technológiák

4.1 Frontend technológiák

A frontend React, TypeScript és Vite használatával készült. A React komponensalapú szemlélete lehetővé teszi, hogy az alkalmazás kisebb, újrafelhasználható felületi egységekből épüljön fel. A ProdCty esetében ilyen egység lehet például az audio track-kártya, a profilnézet, a feltöltési űrlap, a feedback popup, az adminisztrációs panel vagy a navigációs komponens.

A TypeScript használata azért indokolt, mert a frontend többféle adatstruktúrával dolgozik: felhasználókkal, track objektumokkal, kommentekkel, ratingekkel, reportokkal és collab requestekkel. A típusok előzetes meghatározása csökkenti annak esélyét, hogy egy komponens hibás adatmezőt használjon, vagy egy API-válasz rossz szerkezettel kerüljön feldolgozásra.

A Vite gyors fejlesztői szerveret és build folyamatot biztosít. Ez különösen hasznos volt a prototípus-fejlesztés során, mivel a frontend változásai gyorsan ellenőrizhetők voltak böngészőben. A projektben Framer Motion is szerepel, amely finomabb animációk és route transition megoldások megvalósítására használható.

4.2 Backend technológiák

A backend Node.js és Express alapú. Az Express alkalmazás külön route-fájlokba szervezi az autentikációt, a track kezelést, a felhasználói műveleteket, az adminisztrációs funkciókat és az együttműködési kéréseket. Ez a szervezés áttekinthetővé teszi a kódbázist, és lehetővé teszi, hogy az egyes felelősségi körök egymástól elkülönítve legyenek fejleszthetők.

A backend a következő fő feladatokat látja el: HTTP API-végpontok biztosítása, adatok validálása, fájlfeltöltés kezelése, jogosultságok ellenőrzése, MongoDB műveletek végrehajtása, hibaüzenetek visszaadása, valamint adminisztrátori műveletek kezelése. A fájlfeltöltéshez Multer könyvtár került alkalmazásra, a jelszavak hash-eléséhez pedig bcryptjs.

4.3 Adattárolás és hitelesítés

Az adattárolás MongoDB és Mongoose segítségével történik. A dokumentumalapú modell illeszkedik a projekt jellegéhez, mert a feltöltött hanganyagokhoz többféle metaadat kapcsolódhat: cím, leírás, műfaj, BPM, hangnem,

energia, címkék, licencjelölés, audio URL, statisztikák és közösségi interakciók. A Mongoose modellek validációt és egységes adatkezelési réteget biztosítanak.

A hitelesítés JWT alapú. Sikeres bejelentkezéskor a felhasználó tokenet kap, amelyet a frontend a későbbi védett API-hívásokhoz használ. A backend requireAuth middleware segítségével ellenőrzi, hogy a kéréshez tartozik-e érvényes felhasználó. Az adminisztrátori műveletek további szerepkör-ellenőrzést igényelnek, így elkülönülnek a normál felhasználói műveletektől.

4.4 Fejlesztési és ellenőrzési eszközök

A projekt teszteléséhez Jest és Supertest került alkalmazásra. A Jest a JavaScript/Node.js környezetben futó automatikus tesztek kezelésére szolgál, a Supertest pedig HTTP API-végpontok tesztelésére használható. A repóban GitHub Actions alapú CI konfiguráció is szerepel, amely a későbbi automatikus ellenőrzési folyamatok alapja lehet.

A dokumentáció Markdown alapokon készült, és a repóban külön docs könyvtár tartalmazza a projekt dokumentációs munkapéldányait. A README fájl futtatási útmutatót, konfigurációs leírást, tesztelési parancsokat és bemutatási sorrendet is tartalmaz.

5. Alkalmazott módszerek és rendszerterv

5.1 Architektúra

A ProdCty kliens-szerver architektúrában készült. A rendszer három fő logikai rétegre bontható: frontend, backend és adatbázis. A frontend felelős a felhasználói interakciókért és a megjelenítésért, a backend kezeli az üzleti logikát és az API-végpontokat, az adatbázis pedig a tartós adatok tárolását végzi.

A frontend a felhasználó böngészőjében fut, és HTTP-kéréseken keresztül kommunikál a backenddel. A felhasználó például bejelentkezéskor elküldi az azonosító adatait, a backend ellenőrzi azokat, majd sikeres hitelesítés esetén token-t küld vissza. Ezt a token-t a frontend eltárolja, és a későbbi védett műveleteknél felhasználja.

A backend REST API-kat biztosít a fő funkciókhoz. Külön API-csoportok tartoznak az autentikációhoz, a hanganyagok kezeléséhez, a felhasználókhoz, az adminisztrációhoz és az együttműködési kérésekhez. Ez a szétválasztás áttekinthetőbbé teszi a kódot, és megkönnyíti a későbbi bővítést.

5.2 Követelmények

A rendszerrel szemben megfogalmazott fő funkcionális követelményeket az 1. táblázat foglalja össze.

Azonosító	Követelmény	Leírás	Állapot
F1	Regisztráció és bejelentkezés	Felhasználó létrehozása, bejelentkezés, JWT alapú munkamenet.	Megvalósult
F2	Sample library	Sample típusú hanganyagok feltöltése, listázása, szűrése, lejátszása és letöltése.	Megvalósult
F3	Demo board	Demo típusú hanganyagok feltöltése, lejátszása és visszajelzési felület biztosítása.	Megvalósult
F4	Komment és rating	Szöveges visszajelzések, válaszok és 1-5 skálájú értékelés kezelése.	Megvalósult
F5	Reportolás	Problémás upload vagy komment jelentése, admin oldali elbírálással.	Megvalósult
F6	Profilkezelés	Saját és publikus profil, bio, avatar, érdeklődési körök és feltöltések listázása.	Megvalósult
F7	Admin moderáció	Reportok, felhasználók, trackek és kommentek adminisztrátori kezelése.	Megvalósult
F8	Fejlett ajánlórendszer	Személyre szabott ajánlás gépi tanulási vagy részletesebb relevanciaalapú logikával.	Jövőbeli fejlesztés

5.3 Adatmodell

Az adatmodell a rendszer fő entitásait Mongoose modellek formájában reprezentálja. A User modell tárolja a felhasználói adatokat, például a felhasználónevet, email címet, jelszó hash-t, szerepkört, moderációs státuszt, érdeklődési köröket, biográfiát és avatart. A publikus profilnézeteknél fontos, hogy érzékeny adat, például email cím, ne kerüljön visszaadásra.

A Track modell a feltöltött hanganyagokat írja le. A modell tartalmazza a feltöltő adatait, a címet, a tartalomtípust, a műfajt, címkéket, leírást, BPM-et, hangnemet, energia-szintet, aura-megjelenítést, licencjelölést, fájl méretet, MIME-típust, audio URL-t, statisztikákat és létrehozási időt. A kind mező különösen fontos, mert ennek alapján különíthető el a sample és a demo tartalom.

A Comment modell a trackekhez kapcsolódó szöveges visszajelzéseket kezeli. A kommentekhez kategória, időbélyeg, parentId és törlési állapot is kapcsolódhat. A Rating modell egy felhasználó és egy track közötti értékelést tárol, egyedi indexszel biztosítva, hogy egy felhasználó egy trackhez csak egy értékelést adjon. A Report modell problémás feltöltések vagy kommentek jelentését tartalmazza, a CollabRequest modell pedig a demókhoz kapcsolódó együttműködési kéréseket kezeli.

Entitás	Fő szerep	Jellemző mezők
User	Felhasználói profil és jogosultság	username, email, passwordHash, role, moderationStatus, interests, bio, avatarUrl
Track	Feltöltött audio tartalom	title, kind, genre, tags, bpm, audioUrl, licenseLabel, ratingAverage, playCount
Comment	Szöveges visszajelzés	trackId, parentId, userId, author, category, timestampSec, text
Rating	Demo értékelése	trackId, userId, author, score, text
Report	Moderációs jelentés	targetType, trackId, commentId, reporterId, reason, status
CollabRequest	Együttműködési kérés	trackId, requesterId, message, skills, contactPreference, status
TrackVote	Fel/le szavazás	trackId, userId, value, updatedAt

2. táblázat: A rendszer fő adatmodelljei

5.4 API-felépítés

A backend REST API-kat biztosít a fő funkciócsoportokhoz. A munkaverzióban nem szükséges minden végpont teljes részletezése, ezért a dolgozat jelenlegi állapota

csak a fő API-csoportokat emeli ki: autentikáció, track-kezelés, felhasználói profil, közösségi visszajelzések, együttműködési kérések és adminisztrátori moderáció.

5.5 Biztonsági és adatvédelmi szempontok

A rendszer biztonsági szempontból több alapvető megoldást alkalmaz. A jelszavak nem nyílt szöveggént kerülnek tárolásra, hanem hash-elt formában. A védett végpontok requireAuth middleware segítségével ellenőrzik a JWT tokenet. Az admin végpontok további role-alapú ellenőrzést használnak, így egy normál felhasználó nem férhet hozzá moderációs műveletekhez.

A publikus profil és a felhasználókeresés nem ad vissza email címet, mivel az személyes adatnak minősülhet. A collab request funkciónál az Instagram elérés csak elfogadott kérés után válik láthatóvá, ami adatvédelmi szempontból korlátozza a kontaktadatok indokolatlan megosztását. A projekt fejlesztői környezetben lokális fájl tárolást használ, de éles környezetben objektumtár és CDN alkalmazása lenne célszerű.

6. Eredmények és megvalósítás

6.1 Backend megvalósítás

A backend Express alkalmazásként működik. Az alkalmazás induláskor biztosítja a feltöltési könyvtár meglétét, majd beállítja a CORS-kezelést, a JSON kérések feldolgozását, a healthcheck végpontot és az egyes route-csoportokat. Az ismeretlen útvonalak egységes 404-es hibaüzenetet adnak vissza.

A backend fő route-csoportjai az authRoutes, adminRoutes, collabRoutes, trackRoutes és userRoutes fájlokban találhatók. Ez a bontás segíti a karbantarthatóságot, mert az autentikáció, adminisztráció, együttműködés, audio tartalom és felhasználói profil logikája elkülönül. A védett végpontoknál a requireAuth middleware biztosítja a felhasználói azonosítást, az admin végpontoknál pedig a requireAdmin ellenőrzés védi a moderációs funkciókat.

A fájlfeltöltés során a rendszer ellenőrzi a feltöltött audiofájl alapvető tulajdonságait és metaadatait. A feltöltött fájl fejlesztői környezetben lokális uploads könyvtárba kerül, a hozzá tartozó metaadatok pedig MongoDB-ben tárolódnak. A Track dokumentum audioUrl és storagePath mezői kapcsolják össze az adatbázisban tárolt metaadatot a tényleges fájlal.

6.2 Frontend megvalósítás

A frontend React és TypeScript alapokon készült. Az alkalmazás több nézetet biztosít: autentikációs felületet, sample library-t, demo boardot, profilt, publikus profilt, collab inboxot és admin felületet. A frontend API-hívásokkal kommunikál a backenddel, és a bejelentkezést követően a JWT tokenet használja a védett műveletekhez.

A Library nézet célja a sample tartalmak kezelése. Itt a felhasználó böngészheti a feltöltött sample-eket, szűrhet és kereshet, elindíthatja a lejátszást, valamint letöltheti a megfelelő tartalmakat. A Demos nézet ezzel szemben a visszajelzésre feltöltött demókra koncentrál. Itt nagyobb szerepet kap az értékelés, a kommentelés, az aura-megjelenítés és az együttműködési kérés.

A felhasználói profil lehetővé teszi a saját adatok, például avatar, bio és érdeklődési körök kezelését, valamint a saját feltöltések áttekintését. A publikus profilnézet más felhasználók feltöltéseinek és alapvető profilinformációinak megtekintésére szolgál, érzékeny adatok megjelenítése nélkül.

6.3 Sample és demo tartalmak elkülönítése

A rendszer egyik legfontosabb tervezési pontja a sample és demo tartalmak elkülönítése. A sample tartalom elsődlegesen könyvtárszerűen böngészhető, lejátszható és letölthető hanganyagként jelenik meg. A demo tartalom ezzel szemben olyan feltöltés, amelynek célja a visszajelzés, értékelés és potenciális együttműködés.

Ez a különbségtétel a Track modell kind mezőjében jelenik meg, amely sample vagy demo értéket vehet fel. A backend külön listázó végpontokat biztosít a sample és demo tartalmakhoz, a frontend pedig eltérő nézetekben jeleníti meg őket. A letöltési logika szintén ehhez a megkülönböztetéshez kapcsolódik, mivel a demo tartalmak esetén nem a letöltés, hanem a meghallgatás és visszajelzés a hangsúlyos.

6.4 Közösségi funkciók

A közösségi funkciók közé tartozik a kommentelés, a rating, a fel- és leszavazás, a reportolás, valamint az együttműködési kérés. A kommentek lehetővé teszik a részletes szöveges visszajelzést, a rating pedig strukturáltabb, számszerű értékelést ad. A TrackVote modell egyszerű fel- és leszavazási logikát biztosít, amely gyors reakciót tesz lehetővé.

A report funkció célja, hogy a felhasználók jelezni tudják a problémás tartalmakat vagy kommenteket. A jelentéshez indok és opcionális részletes leírás kapcsolódik, amelyet az adminisztrátor később elbírálnak. A CollabRequest funkció a demókhoz kapcsolódó együttműködési lehetőséget támogatja: a felhasználó üzenetet, képességeket és preferált kontaktmódot adhat meg, a track tulajdonosa pedig elfogadhatja vagy elutasíthatja a kérést.

6.5 Adminisztrátori funkciók

A közösségi platform fenntarthatósága szempontjából lényeges az adminisztrátori moderáció. A ProdCty admin felülete moderációs statisztikákat, report listát, felhasználói kezelést, track kezelést és komment kezelést biztosít. Az adminisztrátor elbírálnak a reportokat, törölhet problémás tartalmakat vagy kommenteket, figyelmeztetheti a felhasználókat, illetve szükség esetén tiltást alkalmazhat.

A rendszer figyelmeztetésszámot is tárol, így a többszörösen problémás felhasználók szabályalapú moderációs folyamatban kezelhetők.

7. Tesztelés és értékelés

A fejlesztés során automatikus és manuális tesztelés is szerepet kapott. Az automatikus tesztek célja, hogy a legfontosabb backend-funkciók működése ellenőrizhető legyen. Ilyen funkció például az autentikáció, egyes API-végpontok működése, hibás bemenetek kezelése, jogosultsági esetek vizsgálata, valamint az audio aura generálás és a trackService alaplogikája.

A repóban található JUnit riport alapján a tesztfuttatás 19 tesztet tartalmazott, amelyek 0 hibával és 0 sikertelen teszttel futottak le. A tesztek között szerepel például a publikus user search email cím nélküli visszaadása, a play count explicit növelése, az erősebb jelszóvalidáció, az admin áttekintés jogosultsági ellenőrzése, a második admin figyelmeztetés utáni automatikus tiltás, valamint a track validációs logika ellenőrzése.

A coverage riport szerint az összesített sorlefedettség 34,2 százalék. Ez azt jelenti, hogy az automatikus tesztelés az alapvető működést ellenőrzi, de nem tekinthető teljes lefedettségűnek. A projekt MVP állapotában ez elfogadható kiindulási alap, ugyanakkor a későbbi fejlesztés során érdemes bővíteni a teszteket különösen a jogosultságkezelés, a fájlfeltöltés, az adminisztrátori műveletek, a frontend komponensek és a hibás bemeneti esetek irányába.

7.1 Manuális tesztelési folyamat

A manuális tesztelés során a fő felhasználói folyamatokat kell végigpróbálni. Ide tartozik a regisztráció, a bejelentkezés, a hanganyag feltöltése, listázása, meghallgatása, sample letöltése, demo értékelése, komment írása, jelentés létrehozása, együttműködési kérés küldése, profiladatok módosítása, valamint az adminisztrátori funkciók használata.

A manuális tesztelés különösen fontos az audiolejátszás és a felhasználói felület esetében, mert ezek nem kizárólag adatlogikai kérdések. Ellenőrizni kell, hogy a felhasználó érthető visszajelzést kap-e, a hibák nem eredményeznek-e félrevezető állapotot, és a sample/demo elkülönítés a felületen is egyértelmű-e.

7.2 Ismert korlátok

A jelenlegi rendszer fejlesztői környezetre optimalizált. A médiafájlok lokálisan tárolódnak, ami bemutatási célra megfelelő, de éles rendszerhez objektumtár és CDN réteg lenne szükséges. Az ajánlási logika egyszerű szabályokon és pontozásokon alapul,

nem teljes értékű személyre szabott ajánlórendszer. A vizuális CSS rétegben még vannak olyan elemek, amelyek a korábbi prototípusból származnak, és későbbi refaktorálással tisztíthatók.

A rendszer így is teljesíti az MVP célját: bemutatja az audiofókuszú közösségi platform alapvető működését, a sample és demo tartalmak elkülönítését, a közösségi visszajelzési folyamatokat, valamint az adminisztrátori moderáció alapjait.

8. Mesterséges intelligencia használata a fejlesztés során

A ProdCty fejlesztése során két mesterséges intelligencia alapú eszközt használtam rendszeresen: a ChatGPT-t és a Codexet. Ezeket elsősorban nem a projekt alapötletének vagy céljainak meghatározására, hanem a konkrét fejlesztési munka támogatására használtam. A projekt iránya, célcsoportja és fő koncepciója saját döntés eredménye volt: egy audiofókuszu közösségi platform létrehozása producereknek és zenészeknek, amely sample és demo jellegű hanganyagokat kezel.

A Codexet főként kódírásnál és implementációs részfeladatoknál használtam. Ilyen terület volt például egyes backend route-ok, validációs logikák, segédfüggvények, jogosultságkezelési ellenőrzések és ismétlődő kódrészek elkészítése. A projektben több funkció hasonló API-műveletekre épül, például létrehozásra, listázásra, módosításra, törlésre vagy hibás bemenet kezelésére. Ezeknél a Codex gyakran hasznos kiindulási kódot adott, amelyet utána a projekt meglévő szerkezetéhez kellett igazítani.

A ChatGPT-t inkább magyarázatra, hibakeresésre, refaktorálási ötletekre és dokumentációs megfogalmazásra használtam. Amikor egy hibaüzenet alapján nem volt egyértelmű, hogy a probléma a backend logikából, az adatbázis-kapcsolatból, a frontend API-hívásból vagy a jogosultságkezelésből ered, a ChatGPT segített több lehetséges okot végiggondolni. A kapott válaszokat azonban nem fogadtam el automatikusan, hanem ellenőrzési listaként kezeltem.

Az MI-t leginkább kódírára és kódjavításra használtam. Backend oldalon hasznos volt például a Mongoose modellekhez kapcsolódó validációk, a JWT alapú védett végpontok, az adminisztrátori funkciók és a hibakezelési ágak kialakításánál. Frontend oldalon is adott segítséget komponensekhez, API-hívásokhoz és állapotkezeléshez, de ezen a területen több kézi módosításra volt szükség. A backend logikai feladatoknál általában pontosabb és könnyebben használható kiindulási alapot adott, míg a frontend esetében gyakran sablonosabb, kevésbé egyedi megoldásokat javasolt.

A használat során többször előfordult, hogy egy promptot nem volt elég egyszer megfogalmazni. Ugyanazt a problémát néha többféleképpen kellett leírnom, mire a

válasz valóban használható lett. Gyakran pontosítani kellett, hogy melyik fájlról, milyen adatszerkezetről, milyen meglévő komponensről vagy milyen elvárt működésről van szó. Ez különösen akkor volt fontos, amikor nem különálló kódrészletre volt szükség, hanem egy új funkciót kellett beilleszteni a már meglévő alkalmazáslogikába.

A frontendhez kapcsolódó javaslatokat nagyobb arányban kellett átdolgoznom. Sok esetben a generált komponens technikailag működőképes volt, de nem illeszkedett megfelelően a meglévő designhoz, állapotkezeléshez vagy felhasználói folyamathoz. Emiatt a frontendnél az MI inkább vázlatot adott, míg a végleges szerkezetet, elrendezést és működést kézzel kellett finomítani. Ez alapján azt tapasztaltam, hogy az MI a strukturált backend logikában hasznosabbnak bizonyult, míg a kreatívabb, felhasználói élményre építő frontend megoldásoknál több saját döntést igényelt.

A validálás minden esetben több lépésben történt. Először ellenőriztem, hogy a javasolt kód illeszkedik-e a meglévő fájlstruktúrához, elnevezésekhez és adatmodellhez. Ezután futtatással vizsgáltam, hogy az adott funkció ténylegesen működik-e. Backend esetén API-hívásokkal, frontend esetén böngészőben végzett manuális teszteléssel ellenőriztem az eredményt. Ha a módosítás hitelesítést, feltöltést, kommentelést, reportolást vagy admin funkciókat érintett, külön figyeltem arra, hogy a jogosultságkezelés ne sérüljön.

Többször előfordult, hogy az MI által javasolt megoldás első formájában nem volt megfelelő. Tipikus probléma volt, hogy más mezőnevekkel vagy objektumszerkezettel számolt, mint amit a projekt Mongoose modelljei ténylegesen használtak. Ilyenkor a kód csak akkor vált használhatóvá, ha kézzel hozzáigazítottam a meglévő User, Track, Comment, Rating vagy Report modellhez. Az is előfordult, hogy egy javasolt megoldás működött volna egy egyszerűbb alkalmazásban, de a ProdCty esetében figyelembe kellett venni a sample és demo tartalmak elkülönítését, ezért a logikát módosítani kellett.

Az MI-t nem használtam arra, hogy a projekt alapvető szakmai döntéseit helyettem meghozza. A funkciók prioritizálása, az MVP határainak kijelölése, a sample és demo tartalmak elkülönítésének koncepciója, valamint a végső működési logika saját döntés alapján alakult ki. A projektbe csak olyan megoldás került be, amelyet futtatással és manuális ellenőrzéssel is megfelelőnek ítélttem.

A dokumentáció készítésénél a ChatGPT főként a szövegek szerkezetének kialakításában és a megfogalmazás javításában segített. Itt is fontos volt az ellenőrzés, mert a dokumentációnak pontosan a ténylegesen elkészült alkalmazást kell bemutatnia. Emiatt külön figyelni kellett arra, hogy ne kerüljenek be olyan állítások, amelyek csak jövőbeli fejlesztési lehetőségként szerepelnek, például teljes értékű ajánlórendszer, felhőalapú objektumtárolás vagy éles, nagy terhelésre optimalizált infrastruktúra.

Összességében a ChatGPT és a Codex gyakorlati fejlesztési segédeszközként jelent meg. Gyorsították a munkát, főként ismétlődő kódrészeknél, hibakeresésnél és alternatív megoldások keresésénél, de nem váltották ki a rendszertervezést, a döntéshozatalt, a tesztelést és a validálást. A tapasztalat alapján az MI akkor használható hatékonyan szoftverfejlesztésben, ha a fejlesztő pontosan tudja, mit szeretne elérni, és a kapott eredményt nem kész válaszként, hanem ellenőrizendő munkaverzióként kezeli.

9. Összefoglalás

A szakdolgozatban bemutatott projekt célja egy audiofókuszu közösségi webalkalmazás megvalósítása volt producerek és zenészek számára. A ProdCty olyan platformként készült, amely a hanganyagok feltöltését, meghallgatását, értékelését és kommentelését támogatja, miközben figyelembe veszi a különböző tartalomtípusok eltérő felhasználási szabályait.

A fejlesztés során modern webes technológiák kerültek alkalmazásra. A React és TypeScript alapú frontend biztosítja a felhasználói felületet, a Node.js és Express alapú backend kezeli az üzleti logikát és az API-végpontokat, a MongoDB pedig a rendszer adatainak tárolásáért felel. A JWT alapú hitelesítés lehetővé teszi a védett műveletek szabályozását.

A projekt eredményeként létrejött egy működő MVP, amely bemutatja az audioalapú közösségi platform legfontosabb funkcióit. A rendszer alkalmas további fejlesztések alapjául, például fejlettebb keresés, ajánlórendszer, objektumtárolás, részletesebb moderáció vagy bővített profílfunkciók megvalósítására.

A dolgozat elkészítése során fontos tanulság volt, hogy egy webalkalmazás fejlesztése nem kizárólag kódolási feladat. A sikeres megvalósításhoz szükség van követelmények meghatározására, architektúrais döntésekre, adatmodell-tervezésre, jogosultságkezelésre, hibakezelésre, tesztelésre és dokumentációra is. A ProdCty projekt ezeknek a területeknek az összekapcsolásán keresztül mutatja be egy audiofókuszu közösségi alkalmazás MVP-szintű megvalósítását.

Irodalomjegyzék

- [1] React documentation. <https://react.dev/> Utolsó megtekintés: 2026. április 26.
- [2] TypeScript documentation. <https://www.typescriptlang.org/docs/> Utolsó megtekintés: 2026. április 26.
- [3] Node.js documentation. <https://nodejs.org/en/docs> Utolsó megtekintés: 2026. április 26.
- [4] Express documentation. <https://expressjs.com/> Utolsó megtekintés: 2026. április 26.
- [5] MongoDB Manual. <https://www.mongodb.com/docs/> Utolsó megtekintés: 2026. április 26.
- [6] Mongoose documentation. <https://mongoosejs.com/docs/> Utolsó megtekintés: 2026. április 26.
- [7] Jones, M., Bradley, J., Sakimura, N. 2015. JSON Web Token (JWT). RFC 7519. <https://www.rfc-editor.org/rfc/rfc7519> Utolsó megtekintés: 2026. április 26.
- [8] Jest documentation. <https://jestjs.io/docs/getting-started> Utolsó megtekintés: 2026. április 26.
- [9] OWASP Foundation. OWASP Top 10. <https://owasp.org/www-project-top-ten/> Utolsó megtekintés: 2026. április 26.

Nyilatkozat

Alulírott, Tompa Márton, gazdaságinformatikus BSc szakos hallgató, kijelentem, hogy a szakdolgozatban ismertetettek saját munkám eredményei, és minden felhasznált, nem saját munkából származó eredmény esetén hivatkozással jelöltem annak forrását.

Szeged, 2026. május __.

aláírás

